

ПРИМИ ПРАВИЛЬНОЕ
ТОРГОВОЕ РЕШЕНИЕ!

АНАЛИЗИРУЙ КОТИРОВКИ В METATRADER 5

MetaTrader 5 / Примеры

Разработка показателя качества советников

1

Ricardo Rodrigues Lucca

1 222

Введение

В этой статье мы объясним, как разработать показатель качества, который ваш советник сможет отображать в тестере стратегии. На рисунке 1, приведенном ниже, можно увидеть, что значение «OnTester result» получилось 1,0639375, как пример качества системы, которая была запущена. В этой статье мы узнаем два возможных подхода к расчету качества систем, а также увидим, как вывести оба значения в журнал, поскольку мы можем вернуть только одно из них.



History Quality	100%				
Bars	21610	Ticks	429274	Symbols	1
Initial Deposit	50 000.00				
Total Net Profit	283.00	Balance Drawdown Abs...	142.00	Equity Drawdown Absol...	149.00
Gross Profit	939.00	Balance Drawdown Max...	178.00 (0.36%)	Equity Drawdown Maxi...	205.00 (0.41%)
Gross Loss	-656.00	Balance Drawdown Rela...	0.36% (178.00)	Equity Drawdown Relative	0.41% (205.00)
Profit Factor	1.43	Expected Payoff	6.74	Margin Level	174229.43%
Recovery Factor	1.38	Sharpe Ratio	11.78	Z-Score	0.22 (17.41%)
AHPR	1.0001 (0.01%)	LR Correlation	0.71	OnTester result	1.0639375
GHPR	1.0001 (0.01%)	LR Standard Error	106.70		
Total Trades	42	Short Trades (won %)	27 (55.56%)	Long Trades (won %)	15 (53.33%)
Total Deals	84	Profit Trades (% of total)	23 (54.76%)	Loss Trades (% of total)	19 (45.24%)
	Largest	profit trade	218.00	loss trade	-43.00
	Average	profit trade	40.83	loss trade	-34.53
	Maximum	consecutive wins (\$)	7 (471.00)	consecutive losses (\$)	4 (-154.00)
	Maximal	consecutive profit (count)	471.00 (7)	consecutive loss (count)	-154.00 (4)
	Average	consecutive wins	2	consecutive losses	2

Рисунок 1: Выделено поле "OnTester result".

Запуск торговой модели и создание советника

Прежде чем обратиться к фактору качества системы, необходимо создать базовую систему, которая будет использоваться при тестировании. Мы выбрали простую систему: выберем случайное число и, если оно четное, откроем позицию на покупку; в противном случае мы войдем в позицию на продажу, так как число нечетное.

Для выполнения «жеребьевки» мы будем использовать функцию *MathRand()*, которая предоставит число от 0 (нуля) до 32767. Кроме того, чтобы сделать систему более сбалансированной, мы добавим два взаимодополняющих правила. Таким образом, с помощью этих трех правил мы попытаемся гарантировать более надежную систему. Посмотрите:

- Когда мы не в позиции, мы должны сгенерировать случайное число;
 - Если число 0 или 32767, то мы ничего не будем делать;
 - Если число четное, мы купим количество, соответствующее минимальному размеру лота актива;
 - Если число нечетное, мы продадим объем, соответствующий минимальному размеру лота актива;
- Когда мы находимся в позиции, мы будем перемещать стоп в направлении каждой новой свечи, превышающей предыдущую по направлению движения;
 - Используемый стоп будет основан на индикаторе ATR с периодом 1, нормализованном с помощью EMA 8. Кроме того, он будет расположен в самом отдаленном конце от двух свечей, используемых для анализа;
- Если время выходит за пределы диапазона с 11:00 до 16:00, нам нельзя будет открывать позицию, а в 16:30 позиция должна быть закрыта.

Код, использованный для разработки данных правил, можно увидеть ниже.

```
//--- Используемый индикатор для стопа ATR(1) с EMA(8)...
int ind_atr = iATR(_Symbol, PERIOD_CURRENT, 1);
int ind_ema = iMA(_Symbol, PERIOD_CURRENT, 8, 0, MODE_EMA, ind_atr);
//--- Определяем переменную, которая сообщает о том, что у нас есть сделка...
bool tem_tick = false;
//--- Вспомогательная переменная для открытия позиции.
#include<Trade/Trade.mqh>
#include<Trade/SymbolInfo.mqh>
CTrade negocios;
CSymbolInfo info;
//--- Определяем в OnInit() использование таймера каждую секунду
//--- и запускаем CTrade.
int OnInit()
{
//--- Устанавливаем заполнение, чтобы оставить отложенный ордер до тех пор, пока он не
//--- выполнится полностью.
negocios.SetTypeFilling(ORDER_FILLING_RETURN);
//--- Оставляем фиксированное отклонение, оно не используется в B3.
negocios.SetDeviationInPoints(5);
//--- Определяем символ CSymbolInfo...
info.Name(_Symbol);
//--- Создаем таймер...
EventSetTimer(1);
//--- Определяем основу случайного числа так, чтобы у нас были одинаковые тесты...
MathSrand(0xDEAD);
return(INIT_SUCCEEDED);
}
//--- Поскольку мы определили таймер, мы уничтожим его в OnDeInit().
void OnDeinit(const int reason)
{
EventKillTimer();
}
//--- Функция OnTick только сообщает нам о том, что у нас появилась новая сделка.
void OnTick()
{
tem_tick = true;
}
//+-----+
//| Основная функция советника |
//+-----+
```



```

void OnTimer()
{
    MqlRates cotacao[];
    bool fechar_tudo = false;
    bool negocios_autorizados = false;
    ///--- Есть ли у нас новая сделка?
    if(tem_tick == false)
        return ;
    ///--- Для проверки мы копируем информацию 3 последних свечей....
    if(CopyRates(_Symbol, PERIOD_CURRENT, 0, 3, cotacao) != 3)
        return ;
    ///--- Есть ли у нас новая свеча с момента последней проверки?
    if(tem_vela_nova(cotacao[2]) == false)
        return ;
    ///--- Получаем данные из окна торговли и закрытия...
    negocios_autorizados = esta_na_janela_de_negocios(cotacao[2], fechar_tudo);
    ///--- Если мы собираемся всё закрыть, то если есть позиция, мы закроем ее...
    if(fechar_tudo)
    {
        negocios.PositionClose(_Symbol);
        return ;
    }
    ///--- Если не закрываем всё, то переносим стоп при наличии позиции...
    if(arruma_stop_em_posicoes(cotacao))
        return ;
    if (negocios_autorizados == false) // находимся ли мы за пределами торгового окна?
        return ;
    ///--- Мы находимся в торговом окне, попытаемся открыть позицию!
    int sorteio = MathRand();
    ///--- Правило входа 1.1
    if(sorteio == 0 || sorteio == 32767)
        return ;
    if(MathMod(sorteio, 2) == 0) // Правило "жеребьевки" 1.2 -- четное число - покупка
    {
        negocios.Buy(info.LotsMin(), _Symbol);
    }
    else // Правило "жеребьевки" 1.3 -- нечетное число - продажа
    {
        negocios.Sell(info.LotsMin(), _Symbol);
    }
}
///--- Проверяем наличие новой свечи...
bool tem_vela_nova(const MqlRates &rate)
{
    static datetime vela_anterior = 0;
    datetime vela_atual = rate.time;
    if(vela_atual != vela_anterior) // отличается ли это время от сохраненного?
    {
        vela_anterior = vela_atual;
        return true;
    }
    return false;
}
///--- Проверяем, находится ли время в торговом диапазоне для закрытия позиций...
bool esta_na_janela_de_negocios(const MqlRates &rate, bool &close_positions)
{
    MqlDateTime mdt;

```



```

bool ret = false;
close_positions = true;
if(TimeToStruct(rate.time, mdt))
{
    if(mdt.hour >= 11 && mdt.hour < 16)
    {
        ret = true;
        close_positions = false;
    }
    else
    {
        if(mdt.hour == 16)
            close_positions = (mdt.min >= 30);
    }
}
return ret;
}
//---
bool arruma_stop_em_posicoes(const MqlRates &cotacoes[])
{
    if(PositionsTotal()) // есть позиция?
    {
        double offset[1] = { 0 };
        if(CopyBuffer(ind_ema, 0, 1, 1, offset) == 1 // Удалось ли успешно скопировать EM
            && PositionSelect(_Symbol)) // выберите существующую позицию!
        {
            ENUM_POSITION_TYPE tipo = (ENUM_POSITION_TYPE) PositionGetInteger(POSITION_TYPE);
            double SL = PositionGetDouble(POSITION_SL);
            double TP = info.NormalizePrice(PositionGetDouble(POSITION_TP));
            if(tipo == POSITION_TYPE_BUY)
            {
                if (cotacoes[1].high > cotacoes[0].high)
                {
                    double sl = MathMin(cotacoes[0].low, cotacoes[1].low) - offset[0];
                    info.NormalizePrice(sl);
                    if (sl > SL)
                    {
                        negocios.PositionModify(_Symbol, sl, TP);
                    }
                }
            }
            else // tipo == POSITION_TYPE_SELL
            {
                if (cotacoes[1].low < cotacoes[0].low)
                {
                    double sl = MathMax(cotacoes[0].high, cotacoes[1].high) + offset[0];
                    info.NormalizePrice(sl);
                    if (SL == 0 || (sl > 0 && sl < SL))
                    {
                        negocios.PositionModify(_Symbol, sl, TP);
                    }
                }
            }
        }
        return true;
    }
    // не было никакой позиции

```



```

return false;
}

```

Давайте кратко обсудим приведенный выше код. Мы будем использовать среднее значение, рассчитанное на основе ATR, чтобы определить размер стопов, которые будут размещены на границах свечей, когда мы найдем свечу, которая превзошла предыдущую. Это происходит в функции «*arruma_stop_em_posicoes*». Всегда, когда возвращается значение true, есть одна позиция, и нам не следует продвигаться по основному коду в «*OnTimer*». Я использую эту функцию вместо «*OnTick*», потому что мне не нужно, чтобы длинная функция выполнялась в каждой заключенной сделке, а только для каждой новой свечи определенного периода. Однако в «*OnTick*» переменной присваивается значение true, которое указывает на предыдущую операцию. Это необходимо для того, чтобы тестер стратегии не приостанавливался, когда рынок закрыт, поскольку он будет выполнять функцию, даже если предыдущих сделок не было.

Как видите, до сих пор всё строго соответствовало заданному, включая два определенных окна. Первое — это окно открытия сделок, которое находится между 11:00 и 16:00, а второе — окно управления, которое позволяет алгоритму управлять открытой сделкой, перемещая стоп-ордера до 16:30, в этот момент вы должны закрыть все операции на день.

Обратите внимание, что если мы торгуем этим советником сейчас, «OnTester result» будет равен нулю, как видно на рисунке 2, поскольку мы не предоставили функцию расчета для данного значения.

History Quality	100%				
Bars	2391	Ticks	566565	Symbols	1
Initial Deposit	50 000.00				
Total Net Profit	14.97	Balance Drawdown Abs...	1.47	Equity Drawdown Absol...	4.69
Gross Profit	191.68	Balance Drawdown Max...	35.59 (0.07%)	Equity Drawdown Maxi...	38.74 (0.08%)
Gross Loss	-176.71	Balance Drawdown Rela...	0.07% (35.59)	Equity Drawdown Relative	0.08% (38.74)
Profit Factor	1.08	Expected Payoff	0.14	Margin Level	499953.10%
Recovery Factor	0.39	Sharpe Ratio	1.10	Z-Score	0.82 (58.78%)
AHPR	1.0000 (0.00%)	LR Correlation	-0.27	OnTester result	0
GHPR	1.0000 (0.00%)	LR Standard Error	9.59		
Total Trades	109	Short Trades (won %)	46 (45.65%)	Long Trades (won %)	63 (47.62%)
Total Deals	218	Profit Trades (% of total)	51 (46.79%)	Loss Trades (% of total)	58 (53.21%)
	Largest	profit trade	13.21	loss trade	-12.82
	Average	profit trade	3.76	loss trade	-3.05
	Maximum	consecutive wins (\$)	7 (21.61)	consecutive losses (\$)	7 (-15.26)
	Maximal	consecutive profit (count)	26.23 (5)	consecutive loss (count)	-19.90 (6)
	Average	consecutive wins	2	consecutive losses	2

Рисунок 2: Советник выполнен по паре USDJPY на H1 в режиме 1-минутной OHLC в период с 01.01.2023 по 19.05.2023.

О факторе качества

Чтобы установить значение «OnTester result», нам нужно определить функцию «*OnTester*», которая возвращает значение *double*. Всё так просто! При этом, используя приведенный ниже код, мы получаем результат, показанный на рисунке 3.

History Quality	100%				
Bars	2391	Ticks	566565	Symbols	1
Initial Deposit	50 000.00				
Total Net Profit	14.97	Balance Drawdown Abs...	1.47	Equity Drawdown Absol...	4.69
Gross Profit	191.68	Balance Drawdown Max...	35.59 (0.07%)	Equity Drawdown Maxi...	38.74 (0.08%)
Gross Loss	-176.71	Balance Drawdown Rela...	0.07% (35.59)	Equity Drawdown Relative	0.08% (38.74)
Profit Factor	1.08	Expected Payoff	0.14	Margin Level	499953.10%
Recovery Factor	0.39	Sharpe Ratio	1.10	Z-Score	0.82 (58.78%)
AHPR	1.0000 (0.00%)	LR Correlation	-0.27	OnTester result	1.23
GHPR	1.0000 (0.00%)	LR Standard Error	9.59		
Total Trades	109	Short Trades (won %)	46 (45.65%)	Long Trades (won %)	63 (47.62%)
Total Deals	218	Profit Trades (% of total)	51 (46.79%)	Loss Trades (% of total)	58 (53.21%)
	Largest	profit trade	13.21	loss trade	-12.82
	Average	profit trade	3.76	loss trade	-3.05
	Maximum	consecutive wins (\$)	7 (21.61)	consecutive losses (\$)	7 (-15.26)
	Maximal	consecutive profit (count)	26.23 (5)	consecutive loss (count)	-19.9
	Average	consecutive wins	2	consecutive losses	



Рисунок 3: Советник выполнен по паре USDJPY на H1 в режиме 1-минутной OHLC в период с 01.01.2023 по 19.05.2023.

Следующий код нужно разместить в конце предыдущего кода. В нем мы рассчитываем среднее соотношение риска и доходности сделок: это соотношение обычно выражают как полученную доходность, поскольку риск считается постоянным, равным 1. Следовательно, мы можем интерпретировать соотношение риск/доходность как 1:1,23 или также как 1,23 и (другой пример) 0,43. В первом примере на каждый доллар риска мы получаем 1,23, в то время как во втором — на каждый доллар риска мы теряем 0,43. Следовательно, когда доходность равна 1 или близка к ней, это означает, что у нас ничья, а выше 1 означает, что мы выигрываем.

Так как статистика не дает средней суммы выигрыша или проигрыша, мы используем необработанное значение, нормализованное по количеству сделок с каждой стороны (покупка или продажа). При возвращении значений совершенных сделок прибавляем 1 для их использования, таким образом, если нет сделок с прибылью или убытком, то программа не закроется из-за деления на ноль при расчете. Кроме того, чтобы избежать отображения большого количества числ, как показано ранее на рисунке 1, где было более 5 числ, мы использовали функцию "NormalizeDouble" для отображения результата только с двумя цифрами.

```
double OnTester()
{
//--- Средняя прибыль
double lucro_medio=TesterStatistics(STAT_GROSS_PROFIT)/(TesterStatistics(STAT_PROFIT)+1);
//--- Средние убытки
double prejuizo_medio=-TesterStatistics(STAT_GROSS_LOSS)/(TesterStatistics(STAT_LOSS)+1);
//--- Расчет риска: получаемая доходность
double rr_medio = lucro_medio / prejuizo_medio;
//---
return NormalizeDouble(rr_medio, 2);
}
```

Функция «OnTester» должна присутствовать в каждом коде советника, чтобы значение отображалось в отчете. Чтобы сократить работу по копированию нескольких строк кода, мы вынесем функцию в отдельный файл. Таким образом, нам нужно будет копировать только одну строку за раз. Как можно заметить:

```
#include "ARTICLE_METRICS.mq5"
```



так у нас будет меньше кода! В указанном файле определим функцию с помощью определения. Это необходимо для того, чтобы при желании использовать «include», вы сможете без каких-либо сложностей изменить имя подключаемой функции, избегая возможных ошибок дублирования в случае, если функция «OnTester» уже определена. Таким образом, мы можем рассматривать это как механизм, позволяющий отдать предпочтение «OnTester», который будет вставлен непосредственно в код советника. Если мы хотим использовать его с помощью «include», мы просто закомментируем функцию «OnTester» в коде советника и закомментируем соответствующее определение. Мы вернемся к этому немного позже.

Первоначально файл «ARTICLE_METRICS.mq5» будет выглядеть так:

```
//--- Расчет риска: средняя доходность сделок
double rr_medio()
{
//--- Средняя прибыль
double lucro_medio=TesterStatistics(STAT_GROSS_PROFIT)/(TesterStatistics(STAT_PROFIT)+TesterStatistics(STAT_GROSS_LOSS));
//--- Средние убытки
double prejuizo_medio=-TesterStatistics(STAT_GROSS_LOSS)/(TesterStatistics(STAT_PROFIT)+TesterStatistics(STAT_GROSS_LOSS));
//--- Расчет риска: получаемая доходность
double rr_medio = lucro_medio / prejuizo_medio;
//---
return NormalizeDouble(rr_medio, 2);
}

//+-----+
//| OnTester |
//+-----+
#ifdef SQN_TESTER_ON_TESTER
#define SQN_TESTER_ON_TESTER OnTester
#endif
double SQN_TESTER_ON_TESTER()
{
return rr_medio();
}
```

Обратите внимание, что правильное имя файла должно иметь расширение «mqh». Однако, поскольку я планирую сохранить файл в каталоге советников, я намеренно оставил расширение кода. Всё зависит от нашего выбора.

Первая версия расчета качества

Мы рассмотрим первую версию расчета качества, основанную на подходе, который создал *Санни Харрис*, под названием *Индекс СРС*. В этом расчете используются три показателя, которые перемножаются: риск, средняя доходность, коэффициент успеха и коэффициент прибыли. Однако мы собираемся изменить его, чтобы не использовать коэффициент прибыли и вместо этого использовать наименьшее значение между коэффициентом прибыли и коэффициентом восстановления. Хотя, если мы рассмотрим разницу между ними, нам следует выбрать коэффициент восстановления, я предпочитаю оставить его таким, потому что он уже улучшает расчет.

Следующий фрагмент кода реализует подход, упомянутый в предыдущем абзаце. Нам просто нужно вызвать его в функции «OnTester». Обратите внимание, что мы здесь не добавили 1 к числу сделок, поскольку предоставленное значение является общим, и мы ожидаем, что будет как минимум 1 сделка для оценки.

```
//--- Расчет CPC Index от Sunny Harris
```

```
double CPCIndex()
```

```
{
```

```
double taxa_acerto=TesterStatistics(STAT_PROFIT_TRADES)/TesterStatistics(STAT_TRADES
```

```
double fator=MathMin(TesterStatistics(STAT_PROFIT_FACTOR), TesterStatistics(STAT_REC
```

```
return NormalizeDouble(fator * taxa_acerto * rr_medio(), 5);
```

```
}
```

Второй вариант расчета качества

Второй фактор качества, о котором мы поговорим, называется *Индексом Качества Системы* (SQN, по английский) и был создан Ван Тарпом. Мы проведем расчет сделок, выполняемых каждый месяц, и получим простое среднее значение всех месяцев моделирования. SQN отличается от подхода, описанного в предыдущем разделе, поскольку он стремится подчеркнуть стабильность торговой системы.

Важной особенностью SQN является то, что он штрафует системы, у которых выраженные подъемы и падения. Таким образом, если в системе есть серия мелких сделок и одна большая, то она будет "оштрафована". Это означает, что если у нас есть система с небольшими потерями и большой прибылью, мы будем ее штрафовать. Обратное (маленькие прибыли и большие потери) также будет штрафоваться. Последнее хуже всего для тех, кто торгует.

Помните, что трейдинг – это долгосрочная гонка, и всегда сосредотачивайтесь на том, чтобы следовать своей системе, а не на деньгах, которые заработаете к концу месяца, ведь это слишком изменчиво.

```
//--- стандартное отклонение выполненных сделок по результатам в деньгах
```

```
double dp_por_negocio(uint primeiro_negocio, uint ultimo_negocio,  
double media_dos_resultados, double quantidade_negocios)
```

```
{
```

```
ulong ticket=0;
```

```
double dp=0.0;
```

```
for(uint i=primeiro_negocio; i < ultimo_negocio; i++)
```

```
{
```

```
    //--- try to get deals ticket
```

```
    if((ticket=HistoryDealGetTicket(i))>0)
```

```
    {
```

```
        //--- get deals properties
```

```
        double profit=HistoryDealGetDouble(ticket, DEAL_PROFIT);
```

```
        //--- create price object
```

```
        if(profit!=0)
```

```
        {
```

```
            dp += MathPow(profit - media_dos_resultados, 2.0);
```

```
        }
```

```
    }
```

```
}
```

```
return MathSqrt(dp / quantidade_negocios);
```

```
}
```

```
//--- Расчет System Quality Number, SQN, от Van Tharp
```

```
double sqn(uint primeiro_negocio, uint ultimo_negocio,
```

```
double lucro_acumulado, double quantidade_negocios)
```

```
{
```

```
double lucro_medio = lucro_acumulado / quantidade_negocios;
```

```
double dp = dp_por_negocio(primeiro_negocio, ultimo_negocio,  
lucro_medio, quantidade_negocios);
```

```

if(dp == 0.0)
{
    // Так как стандартное отклонение вернуло нулевое значение, которого мы не ожидал
    // мы его изменяем на average_benefit, так как отклонение отсутствует, что
    // приближает систему к результату 1.
    dp = lucro_medio;
}
//--- Количество сделок здесь будет ограничено на 100, чтобы результат не был
//--- максимизирован из-за большого количества сделок.
double res = (lucro_medio / dp) * MathSqrt(MathMin(100, quantidade_negocios));
return NormalizeDouble(res, 2);
}

//--- возвращается, если найден новый месяц
bool eh_um_novo_mes(datetime timestamp, int &mes_anterior)
{
    MqlDateTime mdt;
    TimeToStruct(timestamp, mdt);
    if(mes_anterior < 0)
    {
        mes_anterior=mdt.mon;
    }
    if(mes_anterior != mdt.mon)
    {
        mes_anterior = mdt.mon;
        return true;
    }
    return false;
}

//--- месячный SQN
double sqn_mes(void)
{
    double sqn_acumulado = 0.0;
    double lucro_acumulado = 0.0;
    double quantidade_negocios = 0.0;
    int sqn_n = 0;
    int mes = -1;
    uint primeiro_negocio = 0;
    uint total_negocios;
    //--- запрашиваем торговую историю
    if(HistorySelect(0,TimeCurrent()) == false)
        return 0.0;
    total_negocios = HistoryDealsTotal();
    //--- для каждой сделки рассчитывается средняя за каждый месяц
    for(uint i=primeiro_negocio; i < total_negocios; i++)
    {
        ulong ticket=0;
        //--- Выбираем нужный тикет для вставки данных
        if((ticket=HistoryDealGetTicket(i))>0)
        {
            datetime time = (datetime)HistoryDealGetInteger(ticket, DEAL_TIME);
            double lucro = HistoryDealGetDouble(ticket,DEAL_PROFIT);
            if(lucro == 0)
            {
                //--- Если результата нет, то мы переходим к следующей сделке.
                continue;
            }
        }
    }
}

```



```

    }
    if(eh_um_novo_mes(time, mes))
    {
        //--- Если у нас есть сделки, то мы вычисляем sqn, иначе он будет равен нул
        if(quantidade_negocios>0)
        {
            sqn_acumulado += sqn(primeiro_negocio, i, lucro_acumulado,
                                quantidade_negocios);

            }
        //--- Вычисленное количество sqns обновляется всегда!
        sqn_n++;
        primeiro_negocio=i;
        lucro_acumulado = 0.0;
        quantidade_negocios = 0;
    }
    lucro_acumulado += lucro;
    quantidade_negocios++;
}
}

//--- при выходе из "for", у нас могут возникнуть нежелательные последствия
if(quantidade_negocios>0)
{
    sqn_acumulado += sqn(primeiro_negocio, total_negocios,
                        lucro_acumulado, quantidade_negocios);

    sqn_n++;
}
//--- берем простую среднюю от sqns
return NormalizeDouble(sqn_acumulado / sqn_n, 2);
}

```



Давайте пройдемся по коду снизу вверх:

Первая функция вычисляет стандартное отклонение набора сделок. Здесь мы следуем рекомендации Ван Тарпа, который подчеркивает включение всех сделок в расчет стандартного отклонения. Однако в окончательной формуле (в функции ниже) мы ограничиваем количество сделок до 100. Это сделано для того, чтобы результат не искажался из-за количества сделок, что делает его более практичным и осмысленным.

Наконец, у нас есть функция *sqn_mes*, которая проверяет, наступил ли новый месяц, и накапливает некоторые данные, необходимые для вышеупомянутых функций. В конце данной функции рассчитывается среднемесячный SQN за период, в котором выполнялось моделирование. Это краткое объяснение предназначено для обзора кода и назначения каждой функции: следуя этому подходу, можно лучше понять расчет SQN.

Функция OnTester может печатать все три значения и может быть запрошена на вкладке тестера или сохранена в файле, или мы даже можем вернуть одно значение, умноженное на другое, чтобы оно появилось в отчете, как можно увидеть ниже.

```

double SQN_TESTER_ON_TESTER( )
{
    PrintFormat("%G,%G,%G", rr_medio(), CPCIndex(), sqn_mes());
    return NormalizeDouble(sqn_mes() * CPCIndex(), 5);
}

```



Перед тем, как завершить

Перед завершением данной статьи, давайте вернемся к теме «включения» и к тому, как избежать ошибки дублирования. Предположим, что у нас есть код советника с функцией «OnTester» и мы хотим установить «include» указанного файла. Это будет выглядеть примерно так, как показано ниже (в этом примере нужно игнорировать содержимое «OnTester»).

```
//+-----+
double OnTester()
{
    return __LINE__;
}
//+-----+
#include "ARTICLE_METRICS.mq5"
```

Данный код приведет к ошибке дублирования функции, поскольку и советник в нашем коде, и включаемый файл имеют функцию с одинаковым именем «OnTester». Однако мы можем использовать два определения, чтобы переименовать одно из них и смоделировать механизм включения или отключения той функции, которую должны использовать. Взгляните на пример ниже.

```
//+-----+
#define OnTester disable
//#define SQN_TESTER_ON_TESTER disable
double OnTester()
{
    return __LINE__;
}
#undef OnTester
//+-----+
#include "ARTICLE_METRICS.mq5"
```

В этом новом формате у нас не будет ошибки дублирования функции, поскольку определение изменит имя функции в коде советника с «OnTester» на «disable». Теперь, если мы закомментируем первое определение и раскомментируем второе, в результате функция внутри файла ARTICLE_METRICS будет переименована в «disable», а функция в файле советника по-прежнему будет называться «OnTester».

Этот подход кажется довольно простым способом для переключения между обеими функциями, не требующим комментирования нескольких строк кода. Хотя это может быть немного более навязчивым, мне кажется, что это может учитываться пользователем. Еще один вопрос, который следует учитывать, — нужно ли нам сохранять эту функцию вместе с советником, поскольку она уже существует во включенном файле, что может привести к путанице.

Заключение

Мы подошли к концу этой статьи, где представляем случайно работающую модель советника, которая послужит примером для демонстрации расчетов качества. Поэтому мы рассмотрим 2 возможных расчета: Ван Тарп и Санни Харрис. Кроме того, мы представляем вводный фактор, используя соотношение риска и доходности. Мы также продемонстрировали, как использование «includes» может облегчить переключение между различными доступными функциями.

Если у вас есть вопросы или вы нашли ошибки, сделайте свои замечания в комментариях к статье! Упомянутые коды советника и файлы метрик находятся в прикрепленном файле.

Используете ли вы какие-либо другие показатели качества? Поделитесь ими, прокомментировав здесь тему статьи, чтобы мы знали об этом! Большое спасибо за внимание.

Следующая статья >>

Ricardo Rodrigues Lucca

[Бразилия](#)

[5619](#)

[Follow us](#) on socials for top
articles and CodeBase updates

